

JAVA PROGRAMMING MASTERCLASS

程式異常的處理

「讓程式不因異常而中止」

[← 返回目錄](#)

Outline

- 第一部分：認識程式錯誤的類別
- 第二部分：處理異常方法
 - try-catch / finally / try-with-resources
 - Throwable 類別方法
 - throw / throws / 自訂異常類別
- 第三部分：自定義錯誤代碼和訊息
- 實作練習

第一部分

認識程式錯誤的類別

程式錯誤的三大類型

錯誤類型	發生時機	說明
語法錯誤 (Syntax Error)	編譯階段	語法不正確，IDE 會警告
語意錯誤 (Semantic Error)	執行後才發現	邏輯錯誤，結果不符預期
執行期間錯誤 (Runtime Error)	程式執行時	語意正確但發生未預期狀況

💡 本章重點：執行期間錯誤，Java 將這類錯誤稱為「異常 (Exception)」

認識簡單的異常實例

除數為 0 的異常：

```
1  public static int myDiv(int x, int y) {  
2      return x / y;  
3  }  
4  public static void main(String args[]) {  
5      System.out.println(myDiv(6, 2));  
6      System.out.println(myDiv(8, 0)); // 異常發生，程式中止  
7      System.out.println(myDiv(9, 4)); // 不會執行到  
8  }
```

⚠ Java 動作：拋出異常訊息並終止程式執行

java.lang.ArithmeticException: / by zero

其他常見的異常

異常類別	觸發情境
<code>NullPointerException</code>	對 <code>null</code> 物件呼叫方法
<code>NumberFormatException</code>	將非數值字串轉換成整數
<code>StringIndexOutOfBoundsException</code>	字串索引超出範圍
<code>ArrayIndexOutOfBoundsException</code>	陣列索引超出範圍
<code>InputMismatchException</code>	使用者輸入的類型錯誤

其他常見的異常 – 範例

```
1 // NullPointerException
2 String str = null;
3 System.out.println(str.length());
4 // NumberFormatException
5 int x = Integer.parseInt("Taipei");
6 // StringIndexOutOfBoundsException
7 String s = "ABCD";
8 char c = s.charAt(10);
```


更清晰的 NullPointerException

特性

說明

具體錯誤訊息

JVM 會詳細指出是哪個變數或方法呼叫回傳了 `null`

```
1 // 若 user 為 null，會明確指出「因為 user 是 null」
2 User user = null;
3 System.out.println(user.getName());
```

💡 除錯更方便：從 Java 14 起，錯誤訊息會直接寫明：`Cannot invoke "User.getName()" because "user" is null`。

第二部分

處理異常方法

傳統防呆方式 – if/else

在可能發生異常的地方用 `if` 預先檢查：

```
1  public static int myDiv(int x, int y) {  
2      if (y == 0) {  
3          System.out.print("除數為0異常發生：");  
4          return 0;  
5      } else {  
6          return x / y;  
7      }  
8  }
```

⚠ 缺點：程式碼冗長、難以維護；異常情境多時效率低落

Java 的處理異常方式

當程式發生異常時，Java 會：

1. 產生對應的異常物件 (**Exception Object**)
2. 在執行緒 (thread) 中搜尋異常處理程式碼
 - 狀況 1：找到異常處理程式碼 → 交給它處理，處理完後可繼續往下執行
 - 狀況 2：找不到 → 往前回溯呼叫鏈，直到 **main** ；若仍找不到，輸出所有異常原因與回溯紀錄，程式中止

異常類別階層 – Throwable

```
1 Throwable
2 |— Error (嚴重系統錯誤，通常不需處理)
3 |   |— OutOfMemoryError
4 |   |— StackOverflowError
5 |— Exception (程式可處理的異常)
6 |   |— RuntimeException (非檢查異常)
7 |     |— ArithmeticException
8 |     |— NullPointerException
9 |   |— IOException (檢查異常)
```

💡 **Throwable** 是所有異常的父類別；所有異常物件均為其子類別或衍生類別的實體

非檢查異常 vs 檢查異常

類型	說明	代表類別
非檢查異常 (Unchecked)	編譯器不強制處理	RuntimeException 及其子類別
檢查異常 (Checked)	編譯器強制要求處理	IOException 、 SQLException 等

💡 檢查異常若未加異常處理，程式在編譯階段就會出現錯誤

try-catch 語法

```
1  try {  
2      // 可能發生異常的敘述  
3  } catch (異常類別 e) {  
4      // 異常處理程式碼 (Exception Handler)  
5  }
```

- **try 區塊**：放置可能發生異常的程式碼；一旦有異常，立即離開 try 進入 catch
- **catch 區塊**：**異常類別** 指定要捕捉的類別；**e** 為異常物件，可取得異常訊息

💡 若 catch 的異常類別不符合實際發生的異常，會跳出 catch，程式繼續往下執行

進入 `catch` 的條件

進入 `catch` 區塊需同時符合兩個條件：

1. `try` 區塊中的程式碼有發生異常
2. `catch` 中定義的異常類別有捕捉到
 - 捕捉到：`catch` 的類別是發生異常的類別，或其父類別

💡 一般實踐：`catch` 裡面用 `Exception` 可捕捉大多數執行期異常

try-catch 範例

```
1  public static String myDiv(int x, int y) {  
2      try {  
3          return Integer.toString(x / y);  
4      } catch (ArithmeticException e) {  
5          System.out.println("除數為0的異常：" + e);  
6          return "執行除法運算時須避開除數為0的";  
7      }  
8  }
```

Throwable 類別的方法

方法	說明
<code>String getMessage()</code>	傳回異常的說明字串
<code>String toString()</code>	傳回異常的完整訊息 (含類別名稱)
<code>void printStackTrace()</code>	回溯顯示程式呼叫的執行過程

Throwable 方法 – 範例

```
1  try {  
2      return Integer.toString(x / y);  
3  } catch (ArithmeticException e) {  
4      System.out.println("異常：" + e);  
5      System.out.println("toString：" + e.toString());  
6      System.out.println("getMessage：" + e.getMessage());  
7      e.printStackTrace();  
8      return "執行除法運算時須避開除數為0的";  
9  }
```

多個 catch 區塊

寫法	說明
多個 <code>catch</code> 區塊	依序匹配，符合即執行對應 catch
單一 <code>catch</code> 捕捉多類別	多個異常類別合併到一個 catch

```
1 catch (IOException e) { ... }  
2 catch (ArithmeticException e) { ... }  
3 // 或：合併寫法  
4 catch (IOException | ArithmeticException e) { ... }
```


多個 catch 區塊 – 範例

```
1  try {
2      FileInputStream fio = new FileInputStream("ABC");
3      System.out.println(5 / 0);
4  } catch (IOException e) {
5      System.out.println("發生 IO 錯誤：" + e);
6      return;
7  } catch (ArithmeticException e) {
8      System.out.println("發生計算錯誤：" + e);
9  }
```

練習 1：多 catch 實作

任務說明

設計一個程式，讀取使用者輸入的 2 個整數，計算除法結果。需能正確捕捉以下兩種異常：

1. 除數為 0 — **ArithmeticException**
2. 輸入非數字 — **InputMismatchException**

練習 1：解題提示

提示說明

1. 宣告 `Scanner` 讀取使用者輸入
2. 在 `try` 區塊內讀取兩個整數並做除法
3. 第一個 `catch` 捕捉 `ArithmeticException`
4. 第二個 `catch` 捕捉 `InputMismatchException`
5. 或改用 `|` 語法合併兩個異常類別到同一 `catch`

捕捉上層的異常

捕捉一個異常類別時，其衍生子類別也可被捕捉。

捕捉的類別	實際捕捉範圍
<code>IndexOutOfBoundsException</code>	含 <code>Array ...</code> 和 <code>String ...</code>
<code>RuntimeException</code>	含所有 <code>RuntimeException</code> 子類別
<code>Exception</code>	含所有非 <code>Error</code> 的異常

⚠ 多個 catch 時，越廣泛的異常類別放越後面，避免遮蔽更具體的 catch

捕捉上層的異常 – 範例

```
1  try {  
2      String str = "Ming-Chi";  
3      char c = str.charAt(3);  
4      System.out.println("c字元是：" + c);  
5      c = str.charAt(10); // 異常發生  
6      System.out.println("c字元是：" + c);  
7  } catch (IndexOutOfBoundsException e) {  
8      System.out.println("索引超出範圍：" + e);  
9  }
```

finally 區塊

不論 try 是否發生異常，**finally** 區塊一定會執行。

```
1  try {  
2      // 可能發生異常的敘述  
3  } catch (異常類別 e) {  
4      // 處理異常  
5  } finally {  
6      // 不論是否異常皆會執行  
7  }
```

💡 常用於：關閉資源（資料庫連線、檔案、網路串流等）

finally 的注意事項

- **finally** 前面必須有 **try** 區塊
- 若 try 無異常：finally 在 try 後執行
- 若 try 有異常且被捕捉：catch 執行完後再執行 finally
- 若 try 有異常但未被捕捉：仍會執行 finally，然後程式中止
- 即使 try 或 catch 有 **return**、**break**、**continue**，finally 仍會執行

try-with-resources

程式會自動關閉在 `try( ... )` 括號內宣告的資源：

```
1  try (Scanner scanner = new Scanner(System.in)) {  
2      System.out.println("請輸入 2 個數字：");  
3      int num1 = scanner.nextInt();  
4      int num2 = scanner.nextInt();  
5  } catch (Exception e) {  
6      System.out.println("發生錯誤：" + e);  
7  }
```

💡 資源必須實作 **AutoCloseable** 介面 (如 Scanner、FileInputStream 等)

try-with-resources 語法增強

特性

說明

使用外部變數

若資源變數是 **final** 或「實質上是 final」，可直接放入 **try(...)** 中

```
1 // 變數在外部宣告，不需在 try() 內重新宣告
2 final Scanner scanner = new Scanner(System.in);
3 try (scanner) {
4     int num1 = scanner.nextInt();
5 } catch (Exception e) {
6     System.out.println(e);
7 }
```

💡 簡化程式碼：從 Java 9 開始，不再需要寫成 **try (Scanner s = scanner)**，程式碼更為簡潔。

練習 2：try-with-resources

任務說明

改寫練習 1，使用 try-with-resources 語法管理 **Scanner** 資源，確保程式結束後 Scanner 自動關閉。不再需要在 **finally** 中手動呼叫 **scanner.close()**。

練習 2：解題提示

提示說明

1. 將 `new Scanner(System.in)` 移入 `try( ... )` 括號內宣告
2. 移除原本 `finally` 中的 `scanner.close()` 呼叫 (若有)
3. `catch` 仍維持捕捉 `ArithmeticException` 和 `InputMismatchException`
4. 觀察程式執行結果是否與原本相同

自行拋出異常 – throw

允許程式設計師自行定義異常發生的條件，然後主動拋出。

語法：

```
1  throw new exception_class("exception message");
```



exception_class : Java 內部或自訂的異常類別

exception message : 自定義的錯誤說明字串

throw — 範例（密碼長度驗證）

```
1  private static void pwdCheck(String pwdStr) throws Exception {  
2      if (pwdStr.length() ≥ 5 && pwdStr.length() ≤ 8) {  
3          System.out.println("密碼驗證成功：" + pwdStr);  
4      } else {  
5          System.out.println("密碼驗證失敗：" + pwdStr);  
6          throw new Exception("密碼長度不符規定");  
7      }  
8  }
```


throw – 再次拋出已捕捉的異常

在 catch 區塊中可以將已捕捉的異常再次拋出：

```
1  public static void main(String[] args) throws IOException {  
2      try {  
3          FileInputStream fio = new FileInputStream("ABC");  
4          System.out.println(5 / 0);  
5      } catch (IOException e) {  
6          throw e;  
7      } catch (ArithmeticException e) {  
8          System.out.println("發生計算錯誤：" + e);  
9      }  
10 }
```


練習 3：throw 實作

任務說明

設計一個密碼檢查程式：

- 密碼長度必須在 5-8 個字元之間
- 準備多組密碼字串，逐一測試
- 長度不符時，使用 **throw** 拋出 **StringIndexOutOfBoundsException**，訊息為「密碼長度不符規定」

練習 3：解題提示

提示說明

1. 建立方法 `pwdCheck(String pwdStr)`，宣告 `throws StringIndexOutOfBoundsException`
2. 長度符合：印「密碼驗證成功」；不符合：`throw new StringIndexOutOfBoundsException("密碼長度不符規定")`
3. 主程式準備字串陣列，用迴圈逐一測試
4. 每次呼叫 `pwdCheck` 用 try-catch 捕捉並印出錯誤訊息

方法拋出異常 – throws

當方法內有多處可能發生異常，可整合為一個方法，並在宣告加上 **throws**：

```
1  public void myMethod() throws 異常類別1, 異常類別2 {  
2      // 可能發生異常的敘述  
3  }
```

- 異常交給**呼叫方**處理
- 呼叫方的程式碼中必須有相對應的 try-catch

⚠ **注意：**原呼叫的程式碼中也要有相對應的異常處理程式，否則會出現編譯錯誤

throw vs throws – 對比

比較項目	throw	throws
位置	方法內部	方法宣告處
用途	主動拋出一個異常物件	宣告此方法可能拋出的異常
語法	<code>throw new XxxException()</code>	<code>void m() throws XxxException</code>
後面接	異常物件 (一次一個)	異常類別 (逗號分隔多個)

```
1 void withdraw(int amt) throws NotEnoughException {  
2     if (amt > balance)  
3         throw new NotEnoughException("餘額不足");  
4 }
```


自訂異常類別

由於所有執行期異常均繼承自 `Exception`，自訂異常類別也必須繼承 `Exception`：

```
1  class 自訂異常類別名稱 extends Exception {  
2      // 定義成員（可以不加任何成員）  
3  }
```

- 繼承後即可使用 `toString()`、`getMessage()` 等方法
- 可加入自訂的成員變數與建構方法

自訂異常類別 – 定義

```
1  @SuppressWarnings("serial")
2  class MyException extends Exception {
3      String str;
4      MyException(String msg) { str = msg; }
5      public String toString() {
6          return ("我定義的MyException發生了 " + str);
7      }
8  }
```


自訂異常類別 – 使用

```
1  try {  
2      System.out.println("try區塊");  
3      throw new MyException("異常訊息");  
4  } catch (MyException e) {  
5      System.out.println("catch區塊");  
6      System.out.println("MyException: " + e);  
7      e.printStackTrace();  
8  }
```

練習 4：自訂異常類別

任務說明

設計一個銀行存提款程式：

- 建立 `NotEnoughException` 繼承 `Exception`，記錄差額 `shortAmount`
- 建立 `MyBank` 類別，有 `deposit()` 和 `withdraw()` 方法
- 提款金額大於存款時，拋出 `NotEnoughException`，傳入差額

練習 4：解題提示

提示說明

1. `NotEnoughException` 加入 `private int shortAmount` 成員與 `getShortAmount()` 方法
2. `MyBank.withdraw()` 判斷 `cashout > balance` 時計算差額
3. `throw new NotEnoughException(差額)`，並在方法宣告加上 `throws NotEnoughException`
4. `main` 中用 try-catch 捕捉，用 `e.getShortAmount()` 列印差額

第三部分

自定義錯誤代碼和訊息

自定義錯誤代碼 – 使用 enum

用 `enum` 定義固定的錯誤代碼與訊息：

```
1  public enum RtnCode {  
2      SUCCESS(200, "Success!! "), SAVE_ERROR(400, "Save error!! ");  
3      private int code; private String message;  
4      RtnCode(int code, String message) {  
5          this.code = code; this.message = message;  
6      }  
7      public int getCode() { return code; }  
8      public String getMessage() { return message; }  
9  }
```

自定義錯誤代碼 – 使用方式

情境

回傳方式

固定代碼 + 固定訊息

`RtnCode.SUCCESS.getCode()` + `.getMessage()`

固定代碼 + 動態訊息

常數 `ERROR_CODE` + `e.getMessage()`

固定代碼 + 動態訊息 – 範例

```
1  public static final int ERROR_CODE = 400;
2  public BaseRes objMapper(String str) {
3      try {
4          Quiz quiz = mapper.readValue(str, Quiz.class);
5      } catch (Exception e) {
6          return new BaseRes(ERROR_CODE, e.getMessage());
7      }
8      return new BaseRes(RtnCode.SUCCESS.getCode(), RtnCode.SUCCESS.getM
message());
9  }
```

練習 5：綜合練習

任務說明

設計一個年齡投票資格檢查系統：

- 準備年齡陣列 `{12, 19, 67}`，逐一取出
- 滿 18 歲：輸出「xx 歲的年齡歡迎投票」
- 未滿 18 歲：拋出自訂異常，訊息為「年齡不符規定」，並輸出「xx 歲的年齡太輕」

練習 5：解題提示

提示說明

1. 建立自訂異常類別，繼承 `StringIndexOutOfBoundsException`
2. 建立 `ageCheck(int age)` 方法，宣告 `throws` 自訂異常類別
3. 方法內：不符合時拋出異常並傳入「年齡不符規定」訊息
4. 主程式用迴圈搭配 try-catch，依結果輸出對應訊息
5. catch 中印出 `e.getMessage()` 顯示錯誤原因

Q & A

程式異常的處理

掌握異常，讓程式更健壯